Qwen2VL Integration Documentation

Overview

The Qwen2VL integration adds advanced multimodal vision-language capabilities to the IOSeiros vision system. This integration enables sophisticated image understanding, natural language descriptions, visual reasoning, and UI analysis using the powerful Qwen2VL model.

Features

Core Capabilities

- **Scene Description**: Generate detailed natural language descriptions of images
- **Object Analysis**: Identify and analyze objects with contextual understanding
- **Text Extraction**: Extract and analyze text content from images
- **UI Analysis**: Analyze user interface elements and interactions
- **Visual Reasoning**: Answer questions about image content using visual reasoning
- **Batch Processing**: Analyze multiple images efficiently

Advanced Features

- **Multimodal Understanding**: Combine visual and textual information
- **Context-Aware Analysis**: Understand relationships between elements
- **Customizable Prompts**: Use custom prompts for specific analysis needs
- **Flexible Configuration**: Support for both API and local inference
- **Integration with Vision System**: Seamless integration with existing vision components

Architecture

```
VisionManager
├── ScreenCaptureManager
├── ImageAnalyzer
├── ObjectDetector
└── Qwen2VLAnalyzer (NEW)
    ├── Scene Description
    ├── Object Analysis
    ├── Text Extraction
    ├── UI Analysis
    └── Visual Reasoning
```

Installation and Setup

Requirements

```
pip install pillow numpy loguru pyyaml
# For API usage:
pip install requests aiohttp
# For local inference:
pip install transformers torch torchvision
```

Configuration

Create a configuration file (`config/qwen2vl_config.yaml`):

```yaml
vision:
  qwen2vl:
    model_name: "Qwen/Qwen2-VL-7B-Instruct"
    max_tokens: 1024
    temperature: 0.1
    enable_scene_description: true
    enable_object_analysis: true
    enable_text_extraction: true
    enable_ui_analysis: true
    enable_reasoning: true

    # Optional API configuration
    api_endpoint: null
    api_key: null
```

Usage Examples

Basic Usage

```python
import asyncio
from eiros_agent.vision import VisionManager
from PIL import Image

async def basic_example():
    config = {
        "vision": {
            "qwen2vl": {
                "model_name": "Qwen/Qwen2-VL-7B-Instruct",
                "max_tokens": 1024,
                "temperature": 0.1
            }
        }
    }

    vision_manager = VisionManager(config["vision"])
    await vision_manager.initialize()

    # Load an image
    image = Image.open("screenshot.png")

    # Analyze with Qwen2VL
    result = await vision_manager.analyze_with_qwen2vl(
        image=image,
        prompt="Describe what you see in this image",
        analysis_type="comprehensive"
    )

    print(f"Analysis: {result['content']}")
    await vision_manager.shutdown()

asyncio.run(basic_example())
```

Screen Analysis

```python
async def screen_analysis():
    vision_manager = VisionManager(config["vision"])
    await vision_manager.initialize()

    # Capture and describe current screen
    result = await vision_manager.describe_screen_with_qwen2vl(
        region=None,  # Full screen
        detail_level="detailed"
    )

    print(f"Screen Description: {result['content']}")
    await vision_manager.shutdown()
```

UI Element Analysis

```python
async def ui_analysis():
    vision_manager = VisionManager(config["vision"])
    await vision_manager.initialize()

    # Analyze UI elements
    result = await vision_manager.analyze_ui_with_qwen2vl(
        image="ui_screenshot.png",
        ui_type="mobile"  # or "web", "desktop", "general"
    )

    print(f"UI Analysis: {result['content']}")

    # Access structured UI data
    if 'ui_elements' in result:
        for element in result['ui_elements']:
            print(f"Element: {element['element_type']} - {element['text']}")

    await vision_manager.shutdown()
```

Visual Reasoning

```python
async def visual_reasoning():
    vision_manager = VisionManager(config["vision"])
    await vision_manager.initialize()

    # Ask questions about an image
    result = await vision_manager.visual_reasoning_with_qwen2vl(
        image="complex_scene.png",
        question="What actions can a user perform in this interface?"
    )

    print(f"Answer: {result['content']}")
    await vision_manager.shutdown()
```

Batch Processing

```python
async def batch_processing():
    vision_manager = VisionManager(config["vision"])
    await vision_manager.initialize()

    images = ["image1.png", "image2.png", "image3.png"]
    prompts = [
        "Describe the main elements",
        "Identify any text",
        "Analyze the color scheme"
    ]

    results = await vision_manager.batch_analyze_with_qwen2vl(
        images=images,
        prompts=prompts,
        analysis_type="comprehensive"
    )

    for i, result in enumerate(results):
        print(f"Image {i+1}: {result['content']}")

    await vision_manager.shutdown()
```

API Reference

VisionManager Methods

`analyze_with_qwen2vl(image, prompt=None, analysis_type="comprehensive")`

Analyze image using Qwen2VL multimodal capabilities.

Parameters:
- `image`: PIL Image, numpy array, or file path
- `prompt`: Optional custom prompt
- `analysis_type`: “comprehensive”, “scene”, “objects”, “text”, “ui”, “reasoning”

Returns: Dictionary with analysis results

`describe_screen_with_qwen2vl(region=None, detail_level="detailed")`

Capture screen and describe it using Qwen2VL.

Parameters:
- `region`: Optional screen region to capture
- `detail_level`: “brief”, “detailed”, “comprehensive”

`analyze_ui_with_qwen2vl(image, ui_type="general")`

Analyze UI elements using Qwen2VL.

Parameters:
- `image`: Image to analyze
- `ui_type`: “mobile”, “web”, “desktop”, “general”

`extract_text_with_qwen2vl(image, language="auto")`

Extract text using Qwen2VL.

`visual_reasoning_with_qwen2vl(image, question)`

Perform visual reasoning based on a question.

`batch_analyze_with_qwen2vl(images, prompts=None, analysis_type="comprehensive")`

Batch analyze multiple images.

Qwen2VLAnalyzer Methods

`analyze_image(image, prompt=None, analysis_type="comprehensive")`

Core analysis method.

`describe_scene(image, detail_level="detailed")`

Generate scene descriptions.

`analyze_objects(image, focus_objects=None)`

Analyze objects in image.

`extract_text(image, language="auto")`

Extract text from image.

`analyze_ui_elements(image, ui_type="general")`

Analyze UI elements.

`visual_reasoning(image, question)`

Perform visual reasoning.

`batch_analyze(images, prompts=None, analysis_type="comprehensive")`

Batch process images.

Configuration Options

Model Configuration

```yaml
qwen2vl:
  model_name: "Qwen/Qwen2-VL-7B-Instruct"  # Model identifier
  max_tokens: 1024                          # Maximum response tokens
  temperature: 0.1                          # Generation temperature
```

API Configuration

```yaml
qwen2vl:
  api_endpoint: "https://api.example.com/v1/chat/completions"
  api_key: "your-api-key"
```

Local Model Configuration

```yaml
qwen2vl:
  local_model:
    device: "auto"              # "cpu", "cuda", "auto"
    precision: "fp16"           # "fp32", "fp16", "int8"
    cache_dir: "./models/qwen2vl"
```

Capability Toggles

```yaml
qwen2vl:
  enable_scene_description: true
  enable_object_analysis: true
  enable_text_extraction: true
  enable_ui_analysis: true
  enable_reasoning: true
```

Response Format

All analysis methods return a structured response:

```
{
    "analysis_type": "comprehensive",
    "timestamp": 1234567890.123,
    "model_info": {
        "name": "Qwen/Qwen2-VL-7B-Instruct",
        "version": "v1.0",
        "processing_time": 2.5
    },
    "confidence": 0.95,
    "content": "Detailed analysis text...",
    "structured_objects": [...],  # For object analysis
    "extracted_text": [...],     # For text extraction
    "ui_elements": [...],         # For UI analysis
    "raw_response": {...}         # Original model response
}
```

Error Handling

```python
from eiros_agent.core.exceptions import VisionError

try:
    result = await vision_manager.analyze_with_qwen2vl(image, prompt)
except VisionError as e:
    print(f"Vision analysis failed: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Performance Considerations

Optimization Tips

1. **Batch Processing**: Use batch methods for multiple images
2. **Image Preprocessing**: Resize large images before analysis
3. **Prompt Optimization**: Use specific prompts for better results
4. **Caching**: Cache results for repeated analyses
5. **Resource Management**: Properly initialize and shutdown components

Memory Management

```
# Proper resource management
async def analyze_with_cleanup():
    vision_manager = VisionManager(config)
    try:
        await vision_manager.initialize()
        # Perform analysis
        result = await vision_manager.analyze_with_qwen2vl(image)
        return result
    finally:
        await vision_manager.shutdown()
```

Integration Examples

iOS App Analysis

```
async def analyze_ios_app():
    # Capture iOS simulator screen
    result = await vision_manager.describe_screen_with_qwen2vl()

    # Analyze UI elements
    ui_result = await vision_manager.analyze_ui_with_qwen2vl(
        image=screenshot,
        ui_type="mobile"
    )

    # Extract any text
    text_result = await vision_manager.extract_text_with_qwen2vl(screenshot)

    return {
        "description": result['content'],
        "ui_elements": ui_result.get('ui_elements', []),
        "text_content": text_result.get('extracted_text', [])
    }
```

Automated Testing

```
async def verify_ui_state(expected_elements):
    screenshot = await vision_manager.capture_screen()

    # Ask specific questions about UI state
    for element in expected_elements:
        question = f"Is there a {element['type']} with text '{element['text']}' vis-
ible?"
        result = await vision_manager.visual_reasoning_with_qwen2vl(
            image=screenshot,
            question=question
        )

        if "yes" not in result['content'].lower():
            return False, f"Missing element: {element}"

    return True, "All elements found"
```

Troubleshooting

Common Issues

1. **Model Loading Errors**
   - Check model name and availability
   - Verify internet connection for downloads
   - Ensure sufficient disk space

2. **API Connection Issues**
   - Verify API endpoint and key
   - Check network connectivity
   - Review rate limits

3. **Memory Issues**
   - Reduce image sizes
   - Use batch processing carefully
   - Monitor system resources

4. **Performance Issues**
   - Optimize prompts
   - Use appropriate analysis types
   - Consider local vs API trade-offs

Debug Mode

```
import logging
logging.basicConfig(level=logging.DEBUG)

# Enable detailed logging
config["vision"]["qwen2vl"]["debug"] = True
```

Future Enhancements

- Multi-image comparison capabilities
- Video analysis support
- Real-time streaming analysis
- Custom model fine-tuning
- Advanced prompt templates
- Performance monitoring and metrics

Contributing

To contribute to the Qwen2VL integration:

1. Fork the repository
2. Create a feature branch
3. Implement your changes
4. Add tests and documentation
5. Submit a pull request

License

This integration follows the same license as the main IOSeiros project.